

KipCalendar API - Полная документация

Добавленный функционал

1 Загрузка расписания из Excel/CSV

Endpoints:

- `POST /api/schedules/upload` - загрузка файла расписания
- `GET /api/schedules/template` - скачать шаблон
- `GET /api/schedules/export/<group_id>` - экспорт расписания группы

Формат файла:

group_name	day_of_week	start_time	end_time	subject_name	teacher_username	room_name	lesson_type
ИС-101	0	09:00	10:30	Математика	teacher1	101	lecture

Использование:

bash

```
curl -X POST http://localhost:5000/api/schedules/upload \  
-H "Authorization: YOUR_TOKEN" \  
-F "file=@schedule.xlsx" \  
-F "organization_id=org_id_123"
```

2 Notifications API

Endpoints:

- `GET /api/notifications` - список уведомлений (параметры: is_read, type, limit, offset)
- `PUT /api/notifications/<id>/read` - отметить как прочитанное
- `PUT /api/notifications/read-all` - отметить все
- `DELETE /api/notifications/<id>` - удалить уведомление
- `DELETE /api/notifications/clear` - удалить все прочитанные
- `GET /api/notifications/settings` - настройки уведомлений
- `GET /api/notifications/pending/<user_id>` - для Telegram бота

- `POST /api/notifications/mark-telegram-sent` - отметка об отправке в TG

Примеры:

javascript

```
// Получить непрочитанные
fetch('/api/notifications?is_read=false', {
  headers: { 'Authorization': token }
})

// Отметить все как прочитанные
fetch('/api/notifications/read-all', {
  method: 'PUT',
  headers: { 'Authorization': token }
})
```

3 Интеграция календаря с журналом

Endpoints:

- `POST /api/calendar/sync-lessons` - синхронизация занятий с календарём
- `GET /api/calendar/user/<user_id>` - полный календарь пользователя

Автоматическое создание событий:

- При создании `actual_lesson` автоматически создаётся событие типа `'lesson'`
- При обновлении занятия (тема, ДЗ) - обновляется событие
- События видны преподавателю и всем студентам группы

Функции в коде:

python

```
create_event_from_actual_lesson(actual_lesson_id) # Создать событие
update_event_from_actual_lesson(actual_lesson_id) # Обновить событие
```

4 Расширенный Users API

Endpoints:

- `GET /api/users/get-by-role?role=teacher&organization_id=...` - пользователи по роли
- `GET /api/users/search?q=иван&organization_id=...&role=student` - поиск
- `GET /api/users/<user_id>/profile` - профиль пользователя
- `PUT /api/users/me/update` - обновить свой профиль
- `POST /api/telegram/link` - привязать Telegram
- `POST /api/telegram/unlink` - отвязать Telegram
- `GET /api/organizations/<org_id>/members` - участники организации

Примеры:

```
javascript

// Поиск преподавателей
fetch('/api/users/get-by-role?role=teacher&organization_id=org123', {
  headers: { 'Authorization': token }
})

// Поиск студента
fetch('/api/users/search?q=Иванов&role=student', {
  headers: { 'Authorization': token }
})
```

5 Telegram Bot

Файл: `backend/telegram_bot.py`

Команды бота:

- `/start` - приветствие и инструкция
- `/link <user_id>` - привязать аккаунт
- `/unlink` - отвязать аккаунт
- `/status` - проверить статус
- `/help` - помощь

Настройка:

1. Создайте бота через @BotFather в Telegram
2. Получите токен
3. В telegram_bot.py замените BOT_TOKEN = "YOUR_TOKEN"
4. Запустите: python telegram_bot.py

Как пользователи подключаются:

1. Открывают бота в Telegram
2. Отправляют /start
3. Копируют User ID из профиля на сайте
4. Отправляют /link 1234567890123456

Автоматическая отправка уведомлений:

- Бот каждые 60 секунд проверяет новые уведомления
 - Отправляет их в Telegram
 - Помечает как отправленные
-

Полная структура API

Authentication

- POST /register - регистрация
- POST /verify - верификация email
- POST /login - вход
- POST /logout - выход
- POST /resend-code - повторная отправка кода
- GET /role - текущая роль
- POST /switch-role - смена роли

Organizations

- POST /api/organizations/create - создать организацию

- POST `/api/organizations/<id>/invitations/create` - создать приглашение
- GET `/api/invitations/<token>` - получить приглашение
- POST `/api/invitations/<token>/accept` - принять приглашение
- GET `/api/organizations/<id>/members` - участники

Buildings & Rooms

- GET/POST `/api/organizations/<id>/buildings` - здания
- PUT/DELETE `/api/buildings/<id>` - управление зданием
- POST `/api/buildings/<id>/rooms` - добавить комнату
- PUT/DELETE `/api/rooms/<id>` - управление комнатой

Groups

- GET/POST `/api/organizations/<id>/groups` - группы
- GET/PUT/DELETE `/api/groups/<id>` - управление группой
- POST/DELETE `/api/groups/<id>/students` - студенты группы

Subjects

- GET/POST `/api/organizations/<id>/subjects` - предметы
- PUT/DELETE `/api/subjects/<id>` - управление предметом
- GET/POST `/api/groups/<id>/subjects` - предметы группы
- PUT/DELETE `/api/group-subjects/<id>` - назначение предмета

Schedule

- GET/POST `/api/groups/<id>/schedule` - расписание
- PUT/DELETE `/api/lessons/<id>` - управление уроком
- POST `/api/schedules/upload` - загрузка из файла
- GET `/api/schedules/template` - шаблон файла
- GET `/api/schedules/export/<id>` - экспорт расписания

Actual Lessons (Journal)

- POST `/api/actual-lessons/generate` - генерация занятий
- GET `/api/actual-lessons/date/<date>/group/<id>` - занятия на дату
- GET/PUT `/api/actual-lessons/<id>` - управление занятием

Marks & Attendance

- POST `/api/marks/add` - добавить оценку
- PUT/DELETE `/api/marks/<id>` - управление оценкой
- GET `/api/marks/student/<id>/subject/<id>` - оценки студента
- POST `/api/attendance/add` - отметить посещаемость
- PUT/DELETE `/api/attendance/<id>` - управление посещаемостью
- GET `/api/attendance/student/<id>` - история посещаемости
- GET `/api/journal/student/<id>/summary` - сводка по журналу

Events (Calendar)

- GET `/events` - список событий
- POST `/api/events/create-plan` - создать план
- POST `/api/events/create-task` - создать задачу
- GET `/event/<username>/<privacy>/<id>` - просмотр события
- PUT/DELETE `/event/<privacy>/<id>` - управление событием
- POST `/api/events/<id>/share` - поделиться событием
- GET/POST `/api/events/<id>/comments` - комментарии
- GET `/api/events/<id>/history` - история изменений

Messenger

- POST `/api/chats/create` - создать чат
- GET `/api/chats` - список чатов
- GET/POST `/api/chats/<id>/messages` - сообщения
- GET `/api/user/search` - поиск пользователей

Notifications

- GET `/api/notifications` - список уведомлений
- PUT `/api/notifications/<id>/read` - отметить прочитанным
- PUT `/api/notifications/read-all` - отметить все
- DELETE `/api/notifications/<id>` - удалить
- DELETE `/api/notifications/clear` - очистить прочитанные

Users

- GET `/api/users/get-by-role` - по роли
 - GET `/api/users/search` - поиск
 - GET `/api/users/<id>/profile` - профиль
 - PUT `/api/users/me/update` - обновить профиль
 - POST `/api/telegram/link` - привязать Telegram
 - POST `/api/telegram/unlink` - отвязать Telegram
-

Запуск системы

Backend

```
bash
cd backend
python app_test.py
```

Telegram Bot

```
bash
cd backend
python telegram_bot.py
```

Frontend

```
bash
```

cd frontend

npm start



TODO (опционально)

- ☐ Rate limiting для API
- ☐ Redis для SocketIO scaling
- ☐ Миграция на PostgreSQL для production
- ☐ Docker compose для всей системы
- ☐ CI/CD pipeline
- ☐ Автоматические тесты
- ☐ Документация Swagger/OpenAPI
- ☐ Мобильное приложение